

Licheng Zheng

(236) 848-2053
Montreal, Canada

musicsir@outlook.com
licheng2018.github.io

Ph.D. candidate focused on ML systems, compiler/runtime analysis, and GPU kernel optimization. Hands-on experience with PyTorch compiler workflows, TorchDynamo/FX/TorchInductor, CUDA/Triton kernels, FlashAttention-style optimization, and rigorous GPU benchmarking. Interested in using LLM agents to automate kernel generation, performance debugging, and ML compiler optimization.

Education Background

École de technologie supérieure (ETS)
Ph.D in Electrical Engineering
University of Southern California (USC)
M.S in Electrical Engineering
Shenyang University of Technology
B.S in Electronic Information Engineering

Montreal, QC, Canada
08/2019-Present
Los Angeles, CA, U.S.A
08/2017-05/2019
Shenyang, Liaoning, China
09/2013-07/2017

TECHNICAL SKILLS

AI Platform / LLM Systems: LLM serving, training/inference workload benchmarking, TTFT, TPOT, throughput, p95 latency, prefill/decode analysis, KV cache, concurrency benchmarking, performance profiling
Distributed Training / GPU Communication: PyTorch FSDP, ZeRO-1/2/3, activation checkpointing, NCCL, AllReduce, ReduceScatter, AllGather, DP/TP/PP, multi-GPU training, communication profiling
ML Compiler / Runtime: PyTorch 2.x compile, TorchDynamo, FX graph, TorchInductor, graph breaks, dynamic-shape guards, SDPA, kernel fusion
GPU Programming / Optimization: CUDA C++, Triton, FlashAttention-style kernels, CUDA events, Nsight Systems, Nsight Compute, PyTorch Profiler, roofline analysis, memory bandwidth modeling
Systems / Deployment: Linux, Docker, Kubernetes fundamentals, GPU resource scheduling, workload profiling, performance prediction
Programming: Python, C++, CUDA C++, PyTorch, Bash

Related Experience

[PyTorch Compiler and AI Compiler Pipeline Project](#)

Built a compiler/runtime benchmarking project and toy AI compiler pipeline to study how PyTorch 2.x compilation, graph capture, operator fusion, and Triton code generation affect GPU performance for ML and attention workloads.

- Evaluated PyTorch **eager mode**, **torch.compile**, **TorchDynamo graph capture**, **FX graph representation**, **AOTAutograd**, and **TorchInductor-generated Triton/CUDA kernels** for representative ML workloads.
- Benchmarked **MLP**, **LayerNorm**, **elementwise**, **naive attention**, and **SDPA attention** cases to compare compilation overhead, cold-start compile time, steady-state latency, kernel fusion benefits, and optimized library dispatch behavior.
- Investigated **graph breaks**, **dynamic-shape guards**, **recompilation triggers**, **Python control flow**, **.item()**, **tensor-dependent branching**, and **small-batch overhead** to explain why compiled execution is not always faster than eager execution.
- Inspected **TorchInductor-generated code** and profiling results to connect compiler decisions with GPU runtime behavior, including kernel launches, memory traffic, and fusion opportunities.
- Built a **toy AI compiler pipeline** with frontend IR construction, graph-level pattern fusion, lowering, and Triton code generation for fused elementwise and MLP-style kernels such as **Linear + ReLU**.
- Demonstrated how **operator fusion** reduces intermediate tensor materialization, global memory movement, and kernel launch overhead, linking frontend graph transformations to backend GPU performance.

[IO-Aware Attention System \(Triton + FlashAttention\)](#)

Designed and implemented IO-aware attention kernels to reduce global memory traffic:

- Modeled attention compute complexity ($O(n^2d)$) vs memory traffic ($O(n^2)$) and identified IO-bound bottleneck
- Built naive Triton attention baseline ($QK^T \rightarrow \text{Softmax} \rightarrow V$) with mask support
- Implemented mini **FlashAttention-style kernel** with:
 - Tiled Q/K loading
 - Online softmax (running max + running sum)
 - SRAM reuse and kernel fusion
- Reduced global memory access through tiling strategy
- Achieved **$\sim 2\times$ speedup vs naive attention implementation with improved global memory efficiency and reduced DRAM traffic.**

[CUDA Kernel Optimization & PyTorch Extension Project](#)

Designed and optimized high-performance CUDA kernels for ML workloads with hardware-aware memory tuning:

- Implemented **Matmul, LayerNorm (fwd/bwd), and numerically stable Softmax** in CUDA C++
- Applied **block tiling, shared memory reuse, loop unrolling, and register-level optimization**
- Optimized kernel occupancy and reduced warp stall by tuning block size and register usage.
- Achieved up to **2–3× speedup over PyTorch** eager baselines on selected memory-bound operators such as LayerNorm and Softmax, and **~200× speedup over CPU baselines** in controlled benchmarks.
- Integrated kernels as **PyTorch C++/CUDA extensions with full autograd support**

[LLM Serving Benchmark & Inference Systems Analysis Project](#)

Designed and benchmarked **LLM inference workloads** to analyze **serving performance, prefill/decode behavior, and concurrency trade-offs** on GPU:

- Built a lightweight **LLM serving benchmark harness** on **T4 GPU** to measure core inference metrics, including **TTFT, TPOT, end-to-end latency, throughput, and p95 latency**, across controlled prompt and output settings.
- Implemented single-request experiments to characterize **prefill vs. decode** behavior and analyzed how **prompt length** impacts **TTFT** while **output length** affects **TPOT** and overall generation latency.
- Developed controlled benchmarking sweeps over prompt length and output length, and generated performance plots such as **TTFT vs. prompt length** and **TPOT vs. output length** to identify distinct bottlenecks between **compute-heavy prefill** and **memory-sensitive decode**.
- Extended the benchmark to **multi-request concurrency experiments** under concurrency levels from **1 to 16**, measuring **throughput scaling, average latency, and tail latency** to study **latency-throughput trade-offs** in inference serving workloads.
- Produced **performance analysis reports** with benchmark curves and system insights on why **throughput saturates** under higher concurrency, why **latency degrades** with request contention, and when **prefill/decode bottlenecks** dominate serving performance.
- Connected workload-level metrics with platform scheduling decisions such as batching, concurrency limits, request queueing, and latency-sensitive inference placement.

[Distributed Training Optimization Project \(FSDP / ZeRO / Activation Checkpointing\)](#)

Designed and optimized large-scale distributed training workflows for billion-parameter language models using PyTorch and NCCL:

- Implemented **FSDP full-shard** training for **GPT-Neo-1.3B** to fit billion-parameter training workloads under constrained multi-GPU memory.
- Analyzed **ZeRO-1/2/3** sharding strategies and mapped their impact on parameter, gradient, and optimizer-state memory footprint.
- Integrated **activation checkpointing** to reduce peak activation memory and support larger effective batch sizes.
- Tuned **microbatch size, gradient accumulation, and sharding** configurations to balance memory usage, GPU utilization, and training stability.
- Profiled **NCCL collectives** and synchronization points to identify communication bottlenecks in distributed training.
- Observed up to **2× training throughput** improvement in controlled experiments after tuning microbatching, checkpointing, and sharding configurations.

[Distributed Training Systems & NCCL Parallelism Project](#)

Designed and optimized distributed training experiments for large-model parallelism with communication-aware benchmarking:

- Built and benchmarked distributed communication workflows on dual-GPU setup using **NCCL collectives**, including **AllReduce, ReduceScatter, and AllGather**, and analyzed bandwidth/latency behavior across message sizes.
- Implemented and compared core large-scale training parallelism strategies, including **Data Parallelism (DP), Tensor Parallelism (TP), and Pipeline Parallelism (PP)**, with hands-on experiments inspired by Megatron-style TP and GPipe-style PP.
- Developed distributed benchmarks to measure communication overhead, scalability, and throughput under different parallel execution patterns and workload granularities.
- Optimized NCCL communication efficiency by improving message granularity, reducing unnecessary synchronization, and refining communication patterns, achieving **25–30% communication overhead reduction** in the project benchmark.
- Produced performance analysis reports with bandwidth curves, latency trends, and parallelism trade-off summaries to explain when **TP/PP/DP** are beneficial under different system constraints.

LLM Quantization Benchmark & Inference Optimization Project

- Built an **LLM quantization benchmark pipeline** to compare **FP16 inference with FP8, INT8, AWQ, and GPTQ** models under consistent prompt, output-length, batch/concurrency, and hardware settings.
- Measured **TTFT, TPOT, throughput, end-to-end latency, p95 latency, GPU memory footprint**, and **generation consistency** under fixed workload settings.
- Analyzed how reduced **weight/activation precision** affects prefill/decode latency, KV-cache memory pressure, and serving throughput.
- Identified scenarios where quantized inference improved serving efficiency, as well as cases where dequantization overhead, unsupported kernels, or quality degradation reduced the practical benefit.
- Generated deployment-oriented analysis to guide quantization choices for memory-constrained and latency-sensitive LLM serving workloads.

System-Level Optimization with Multi-Agent RL (5G / Resource Scheduling)

- Modeled multi-cell interference and resource allocation as a multi-agent optimization problem
- Designed MADDPG / Actor-Critic frameworks for distributed power scheduling
- Built simulation environments for large-scale communication system optimization
- Focused on system-level efficiency, scalability, and coordination under partial observability

Kaggle Google - Fast or Slow? Predict AI Model Runtime(Ranking: 30/616, Top 5%)

- Designed GCN-based performance predictor with graph-structured embeddings

Previous Employment

Research Assistant(full time)	
ETS – Montreal, Quebec, Canada – Sychromedia	09/2019- Present
• Designed communication-aware optimization frameworks for UAV-assisted 5G and LEO satellite systems, integrating multi-agent DRL for distributed scheduling under dynamic topology.	
Mitacs Research Intern	
Ciena – Montreal, Quebec, Canada – Self-Optimizing Fabric (SOF) project	09/2019- 08/2022
• Investigated ML-based optimization methods for self-optimizing network fabric systems.	• Developed simulation and analysis workflows for resource management and system performance evaluation.
• Presented technical progress and research findings to industrial and academic collaborators.	
Mitacs Research Intern	
Vmware – Montreal, Quebec, Canada	08/2022 – 04/2024
• Developed reinforcement-learning-based optimization algorithms for resource allocation and system-level decision making.	• Built experimental workflows to evaluate performance under dynamic workloads and changing system conditions.
• Published related research outcomes in peer-reviewed IEEE venues.	
AICT-Centech Research Intern	
Dromii – Montreal, Quebec, Canada	09/2025 – 12/2025
• Developed computer vision algorithms for pothole detection and classification.	• Developed data augmentation pipelines for road-scene robustness.

Publications

Selected Publications

- IEEE JSAC, 2026: Joint Airborne Wireless Positioning and Communication Services with PDMA.
- IEEE ICC, 2024: Joint positioning and data communication provisioning using 5G PDMA airborne.
- IEEE WCNC, 2022: Interference-aware framework for multi-cell NOMA UAV airborne provisioning.